LOGISTIC REGRESSION

Q1. Data Loading & Understanding

a) Load the dataset into Python

 # Load dataset

loan_data = pd.read_excel('loan_approval.xlsx')

 # Display first 5 rows

loan_data.head()

b) Display the first 5 rows

Output

name	*city*	*income*	*credit_score*	*loan_amount*	*years_employed*	*points*	*loan_approved*
Allison Hill	*East Jill*	*113810*	*389*	*39698*	*27*	*50*	*False*
Brandon Hall	*New Jamesside*	*44592*	*729*	*15446*	*28*	*55*	*False*
Rhonda Smith	*Lake Roberto*	*33278*	*584*	*11189*	*13*	*45*	*False*
Gabrielle Davis	*West Melanieview*	*127196*	*344*	*48823*	*29*	*50*	*False*
Valerie Gray	*Mariastad*	*66048*	*496*	*47174*	*4*	*25*	*False*

---

c) Check the shape of dataset

loan_data.shape

Output

(2000, 8)

Interpretation

The dataset contains 2000 rows and 8 columns.

---

d) Display data types of each column

loan_data.dtypes

Output

Column Name	*Data Type*
name	*object*

city	***object***
income	***int64***
credit_score	***int64***
loan_amount	***int64***
years_employed	***int64***
points	***int64***
loan_approved	***bool***

Q2. Data Cleaning

a) Check missing values

loan_data.isnull().sum()

Output

name 0

city 0

income 0

credit_score 0

loan_amount 0

years_employed 0

points 0

loan_approved 0

Interpretation

No missing values are present in the dataset.

---

b) Handle missing values appropriately

Since there are no missing values, no treatment is required.

---

c) Identify categorical columns

loan_data.select_dtypes(include='object').columns

Output

Index(['name', 'city'], dtype='object')

Interpretation

The categorical columns are:

- ***name***
- ***city***

Q3. Exploratory Data Analysis (EDA)

a) Plot the distribution of target variable

```
sns.countplot(x='loan_approved', data=loan_data)
plt.title('Loan Approval Distribution')
plt.show()
```

Interpretation

The graph shows the distribution of approved and rejected loans.

---

b) Relationship between numerical feature and target variable

```
sns.boxplot(x='loan_approved', y='income', data=loan_data)
plt.title('Income vs Loan Approval')
plt.show()
```

Interpretation

Applicants with higher income are more likely to get loan approval.

---

c) Relationship between Years of Employment and target variable

```
sns.boxplot(x='loan_approved', y='years_employed', data=loan_data)
plt.title('Years Employed vs Loan Approval')
plt.show()
```

Interpretation

People with more years of employment generally have higher chances of loan approval.

Q4. Outlier Detection and Treatment

a) Detect outliers using IQR method

```
numerical_columns = ['income', 'credit_score', 'loan_amount',
'years_employed', 'points']

for col in numerical_columns:
Q1 = loan_data[col].quantile(0.25)
Q3 = loan_data[col].quantile(0.75)
IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR
```

outliers = loan_data[(loan_data[col] < lower_limit) |

(loan_data[col] > upper_limit)]

print(col, 'Outliers:', len(outliers))

---

b) Treat outliers using capping

for col in numerical_columns:

Q1 = loan_data[col].quantile(0.25)

Q3 = loan_data[col].quantile(0.75)

IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR

upper_limit = Q3 + 1.5 * IQR

loan_data[col] = np.where(loan_data[col] > upper_limit,

upper_limit,

loan_data[col])

loan_data[col] = np.where(loan_data[col] < lower_limit,

lower_limit,

loan_data[col])

Interpretation

Outliers were treated using capping to reduce the impact of extreme values.

Q5. Convert Target Variable into Numerical Format and Drop Unnecessary Columns

Convert categorical variables

label_encoder = LabelEncoder()

loan_data['name'] = label_encoder.fit_transform(loan_data['name'])

loan_data['city'] = label_encoder.fit_transform(loan_data['city'])

Convert target variable

loan_data['loan_approved'] = loan_data['loan_approved'].astype(int)

Interpretation

Categorical values are converted into numerical format so that the machine learning model can process them.

Q6. Feature Selection and Data Splitting

a) Separate independent and dependent variables

```
X = loan_data.drop('loan_approved', axis=1)
y = loan_data['loan_approved']
```

---

b) Split dataset into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(
X, y,
test_size=0.20,
random_state=42
)
```

Interpretation

80% of data is used for training and 20% for testing.

Q7. Apply Feature Scaling using StandardScaler

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Interpretation

Feature scaling standardizes the data and improves model performance.

Q8. Logistic Regression Model Building

a) Train Logistic Regression model

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

---

b) Predict output for test data

```
y_pred = model.predict(X_test)
```

Regression Equation

The Logistic Regression equation is:

$$Y = b0 + b1x1 + b2x2 + b3x3 + \ldots + bnxn$$

Error! Filename not specified.

For this model:

$$Y = -0.516 + (-0.0118)(name) + (0.0067)(city) + (-0.291)(income)after\ scaling + (0.0377)(credit\ score) + (-0.065)(loan\ amount) + (-0.017)(years\ employed) + (10.321)(points)$$

Error! Filename not specified.

Interpretation of Coefficients

Feature	***Interpretation***
Income	***Higher income slightly affects loan approval probability***
Credit Score	***Higher credit score increases approval chances***
Loan Amount	***Higher loan amount may reduce approval probability***
Years Employed	***Stable employment influences approval***
Points	***Strong positive impact on loan approval***

Q9. Model Evaluation – Confusion Matrix

a) Generate confusion matrix

```
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

Output

```
[[217 0]
[ 0 183]]
```

---

b) Interpret the results

```
accuracy = accuracy_score(y_test, y_pred)
print('Accuracy:', accuracy)
```

Output

Accuracy: 1.0

Interpretation

The model achieved 100% accuracy on the test dataset.

Classification Report

```
print(classification_report(y_test, y_pred))
```

Interpretation

Precision, Recall, and F1-score are excellent for both classes.

Q10. Model Evaluation – ROC Curve & AUC

a) Plot ROC Curve

```
y_prob = model.predict_proba(X_test)[:,1]


fpr, tpr, threshold = roc_curve(y_test, y_prob)


plt.plot(fpr, tpr)
plt.plot([0,1], [0,1], linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.show()
```

---

b) Calculate AUC Score

```
auc_score = roc_auc_score(y_test, y_prob)
print('AUC Score:', auc_score)
```

Output

```
AUC Score: 1.0
```

Interpretation

An AUC score of 1.0 indicates perfect classification performance.

Final Conclusion

In this project, a Logistic Regression model was successfully developed to predict loan approval status.

Key Findings

- ***No missing values were found in the dataset.***
- ***Outliers were treated using the IQR capping method.***
- ***Feature scaling improved the efficiency of the model.***
- ***Logistic Regression performed extremely well on the dataset.***
- ***The model achieved:***
    - ***Accuracy = 100%***
    - ***AUC Score = 1.0***

Overall Result

The model can effectively predict whether a loan application will be approved or rejected based on customer details.

Google Colab Notebook Structure

1. ***Import Libraries***
2. ***Load Dataset***
3. ***Data Understanding***
4. ***Data Cleaning***
5. ***Exploratory Data Analysis***
6. ***Outlier Treatment***
7. ***Feature Engineering***
8. ***Train-Test Split***
9. ***Feature Scaling***
10. ***Logistic Regression Model***
11. ***Model Evaluation***
12. ***Conclusion***

Thank You